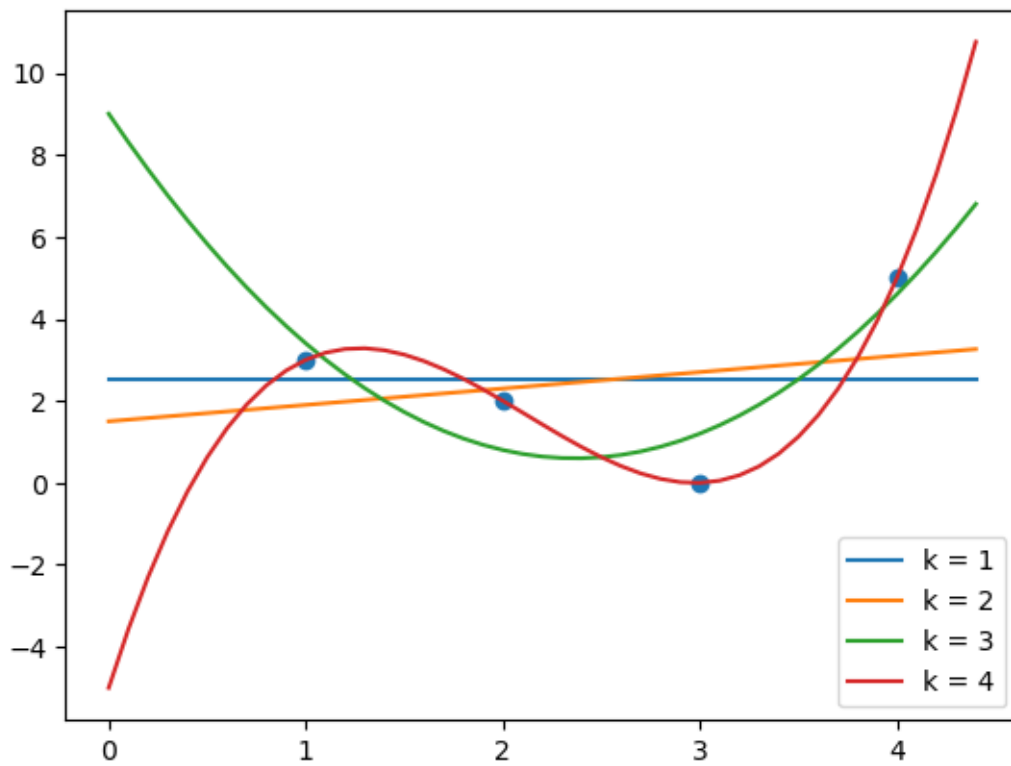


COMP0078 Coursework1

November 18, 2024

1 Q1

1.1 (a)



1.2 (b)

$$k = 1$$

$$\hat{y} = 2.5$$

$$k = 2$$

$$\hat{y} = 0.4x + 1.5$$

$$k = 3$$

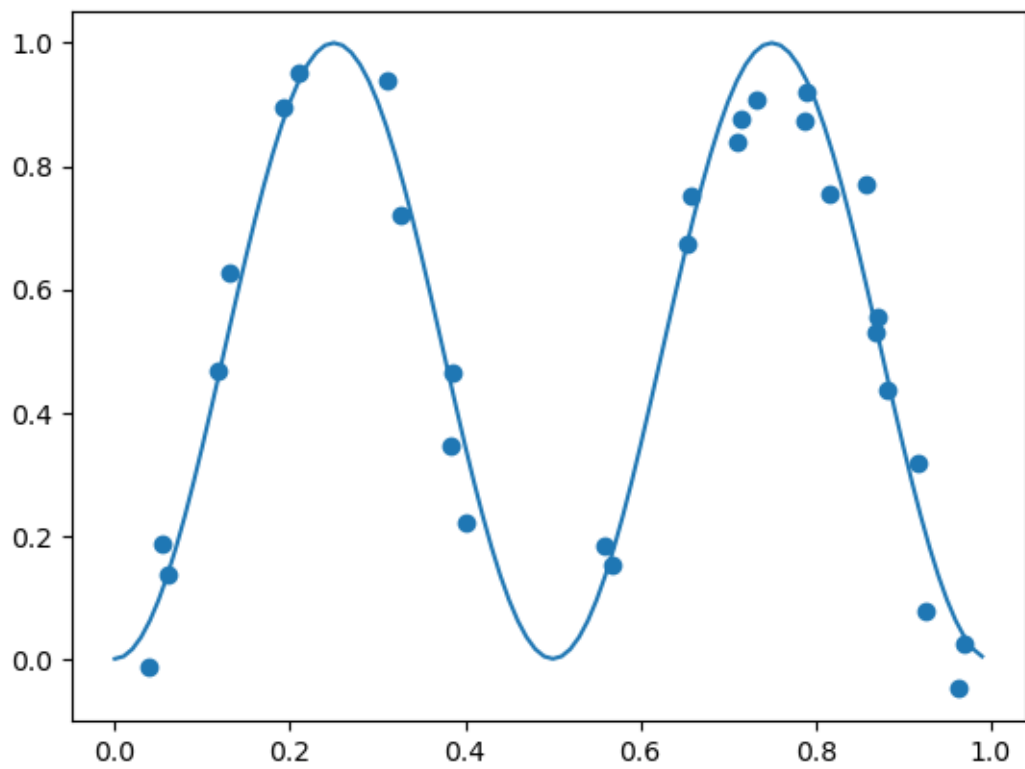
$$\hat{y} = 1.5x^2 - 7.1x + 9.0$$

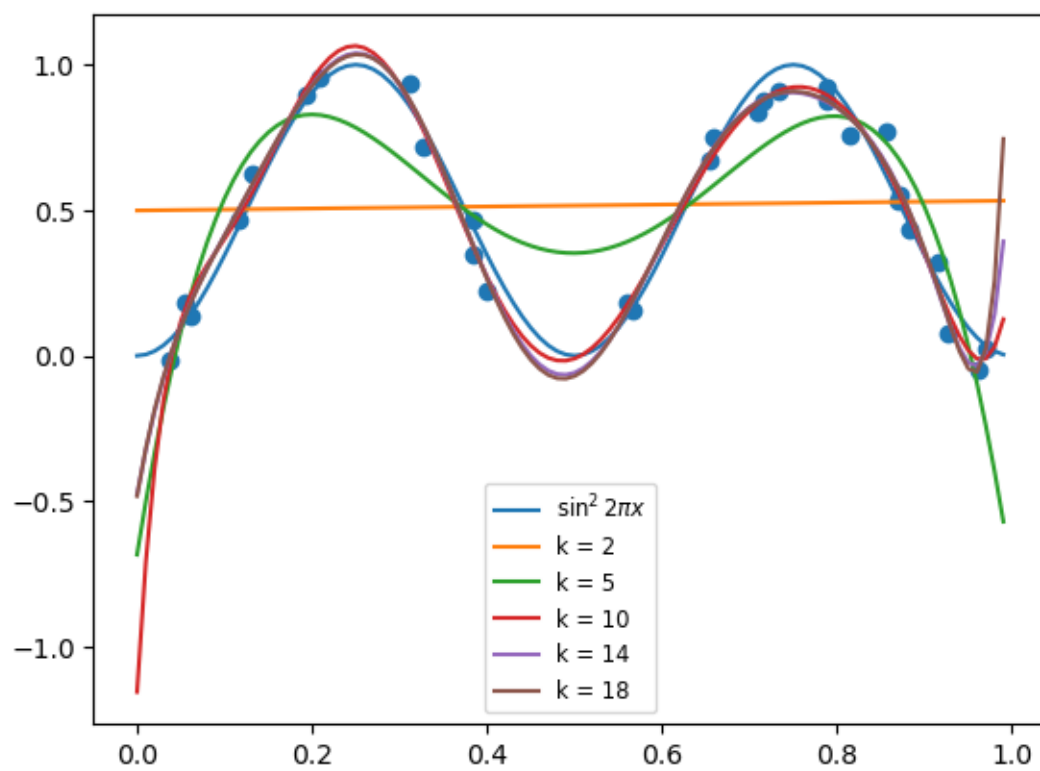
1.3 (c)

k	MSE
1	3.25
2	3.05
3	0.8
4	1.90115e-28

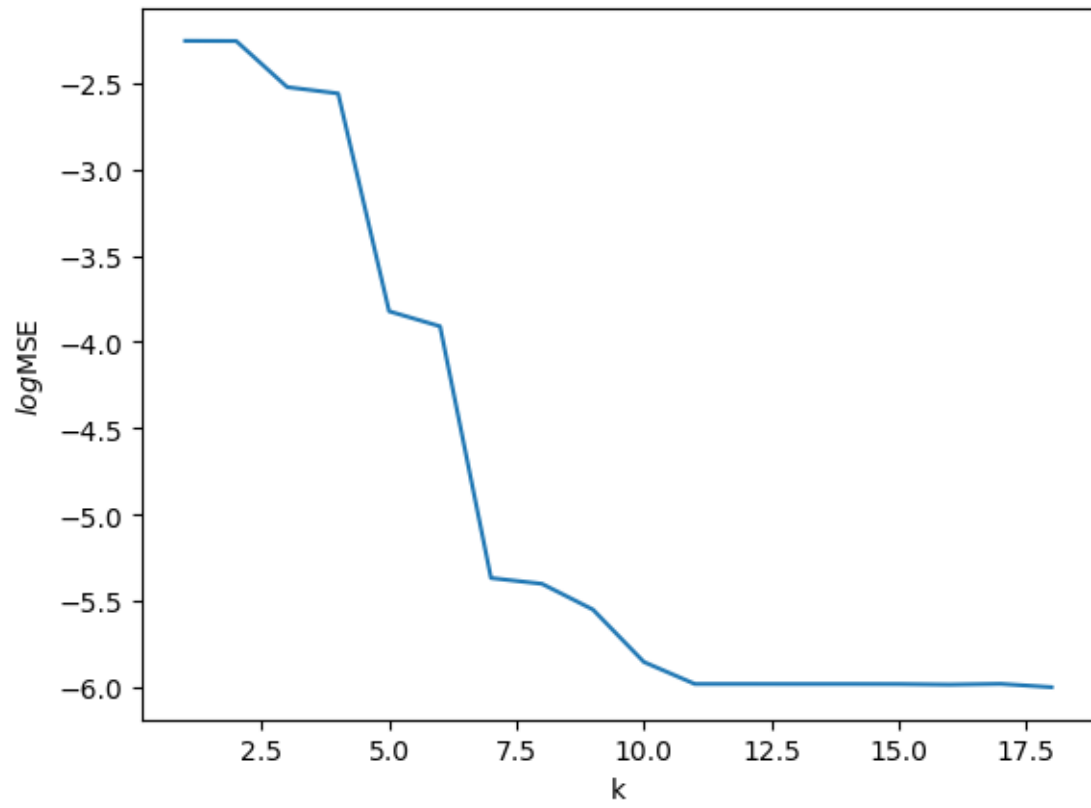
2 Q2

2.1 (a)

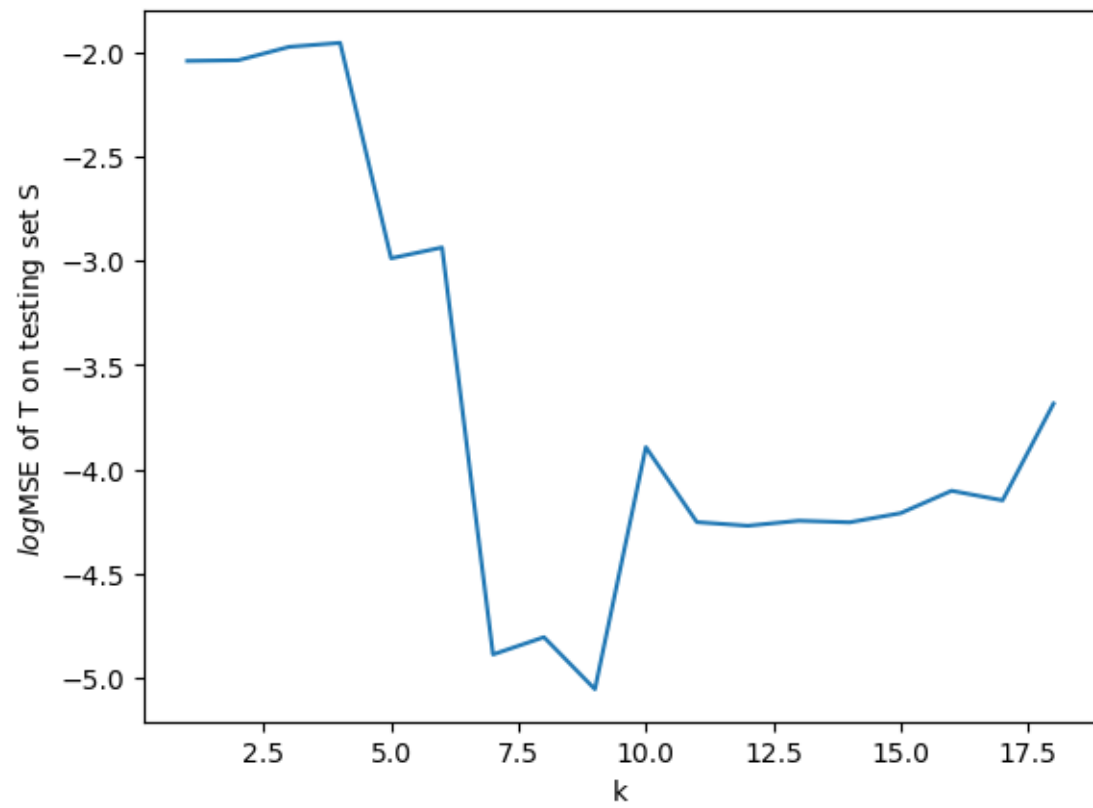




2.2 (b)



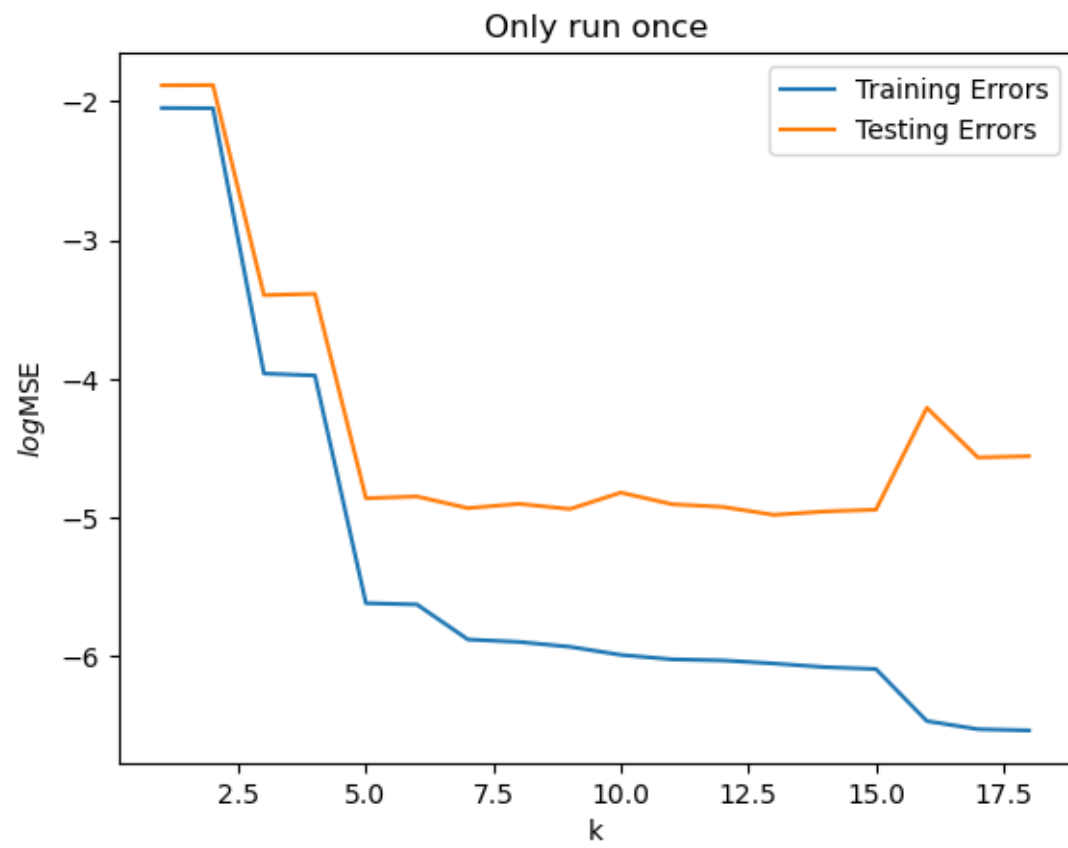
2.3 (c)

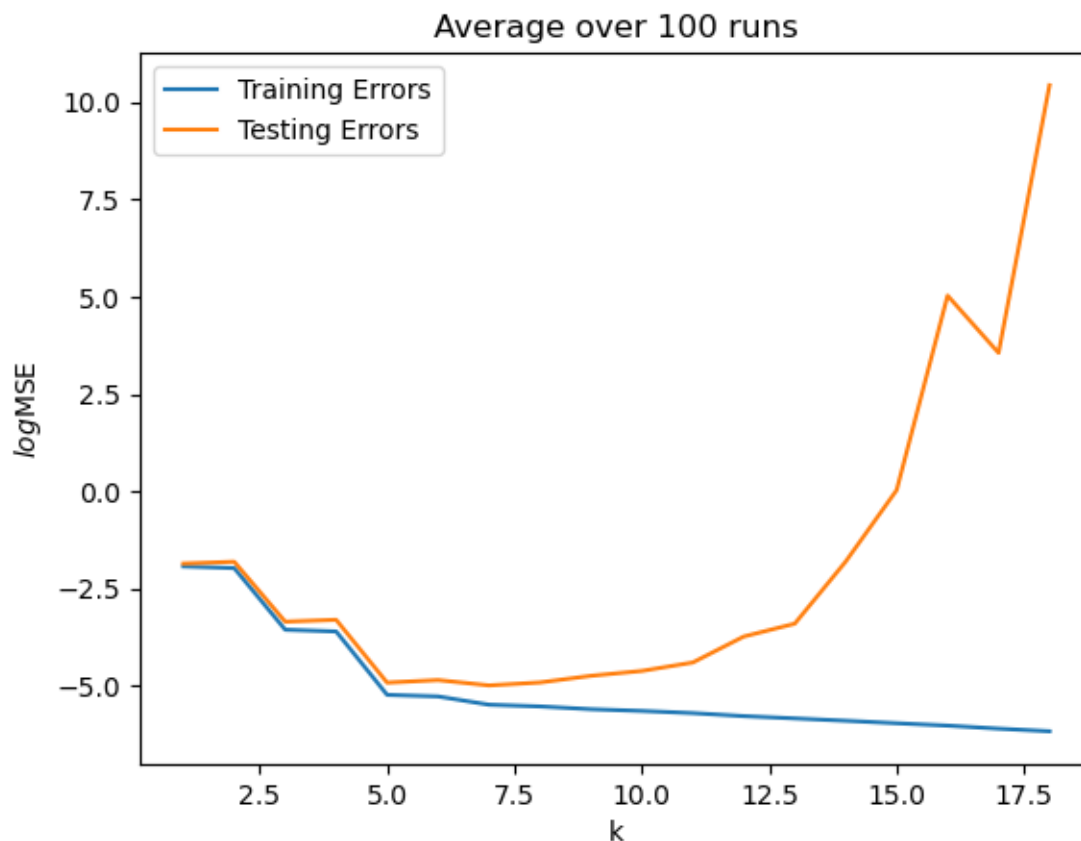


2.4 (d)



3 Q3





4 Q4

Training MSE for constant function is 84.07, testing MSE is 85.39
 Training MSE for single attribute CRIM is 70.94, testing MSE is 74.00
 Training MSE for single attribute ZN is 73.07, testing MSE is 74.64
 Training MSE for single attribute INDUS is 64.58, testing MSE is 65.26
 Training MSE for single attribute CHAS is 81.29, testing MSE is 83.53
 Training MSE for single attribute NOX is 68.63, testing MSE is 70.09
 Training MSE for single attribute RM is 41.74, testing MSE is 47.92
 Training MSE for single attribute AGE is 71.94, testing MSE is 73.72
 Training MSE for single attribute DIS is 78.76, testing MSE is 80.32
 Training MSE for single attribute RAD is 71.38, testing MSE is 74.06
 Training MSE for single attribute TAX is 65.19, testing MSE is 67.76
 Training MSE for single attribute PTRATIO is 61.28, testing MSE is 65.80
 Training MSE for single attribute LSTAT is 38.01, testing MSE is 39.74
 Training MSE for all features is 21.17, testing MSE is 26.56

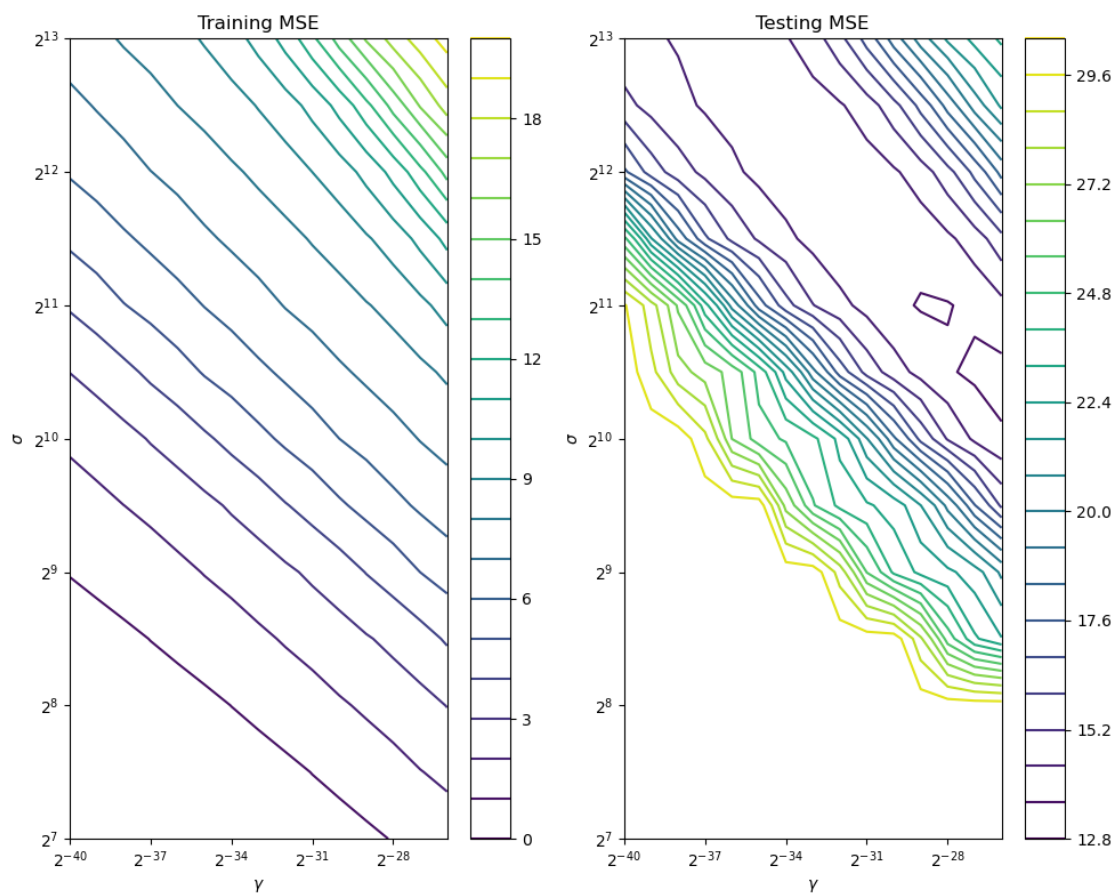
Using a const function to do the regression is effectively calculating

$$w^* = \arg \min_w \sum_i^m (y_i - w)^2 = \frac{1}{m} \sum_i^m y_i$$

w^* is effectively the mean estimate of $\{y_i\}$.

From the output above, we can see the regression on all features outperforms any other single attribute or const function.

5 Q5



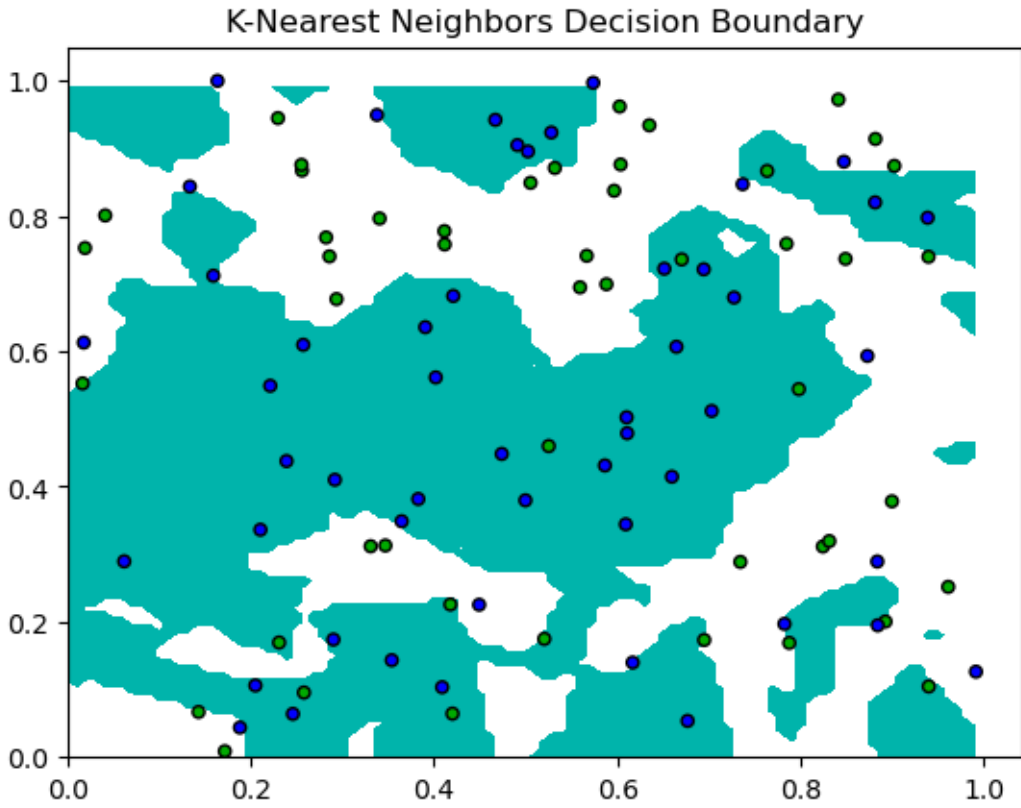
MSE on training set is 8.52. MSE on testing set is 12.73.

Best parameters are:

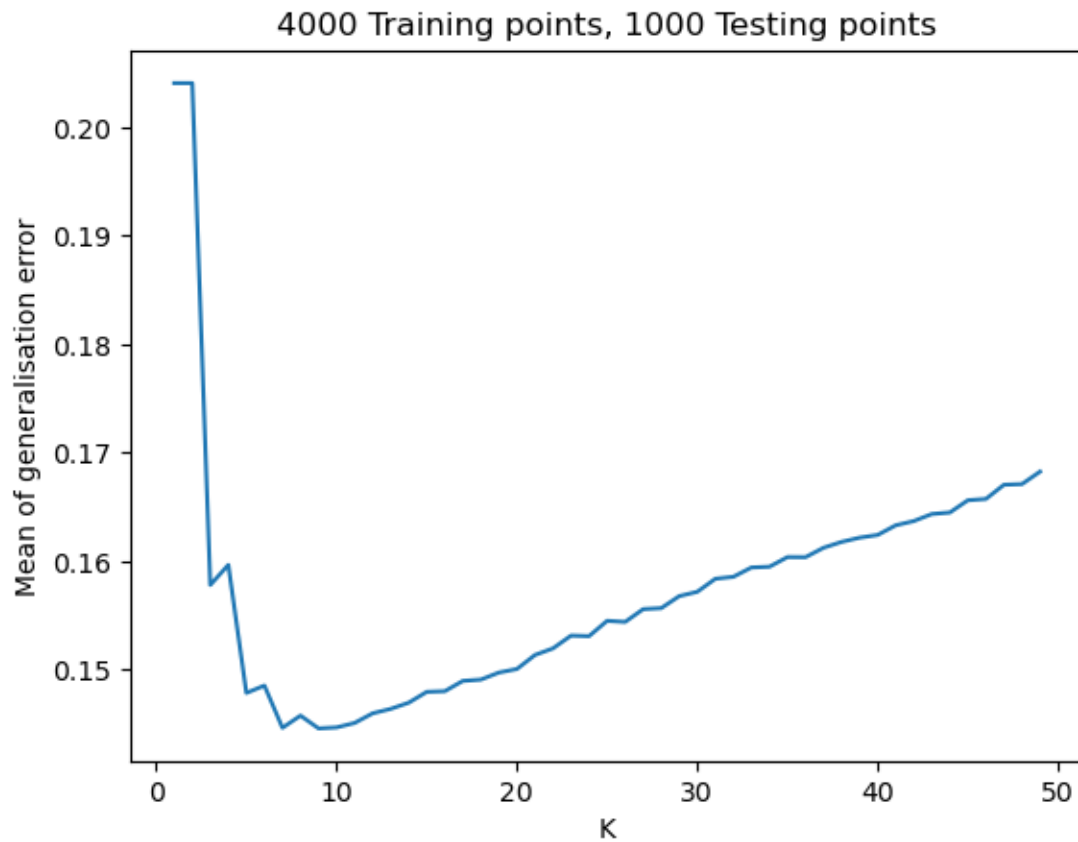
$\gamma : 2^{-26.0}, \sigma : 2^{10.5}$.

Method	MSE train	MSE test
Naive Regression	84.07 ± 6.20	85.39 ± 12.41
Linear Regression (attribute CRIM)	70.94 ± 6.00	74.00 ± 12.42
Linear Regression (attribute ZN)	73.07 ± 5.48	74.64 ± 10.98
Linear Regression (attribute INDUS)	64.58 ± 5.39	65.26 ± 10.96
Linear Regression (attribute CHAS)	81.29 ± 5.96	83.53 ± 12.08
Linear Regression (attribute NOX)	68.63 ± 5.69	70.09 ± 11.60
Linear Regression (attribute RM)	41.74 ± 3.98	47.92 ± 8.20
Linear Regression (attribute AGE)	71.94 ± 5.92	73.72 ± 11.85
Linear Regression (attribute DIS)	78.76 ± 6.20	80.32 ± 12.48
Linear Regression (attribute RAD)	71.38 ± 6.11	74.06 ± 12.46
Linear Regression (attribute TAX)	65.19 ± 5.86	67.76 ± 12.00
Linear Regression (attribute PTRATIO)	61.28 ± 4.76	65.80 ± 9.59
Linear Regression (attribute LSTAT)	38.01 ± 3.36	39.74 ± 6.78
Linear Regression (all attributes)	21.17 ± 2.30	26.56 ± 5.36
Kernel Ridge Regression	8.28 ± 1.75	13.13 ± 2.67

6 Q6

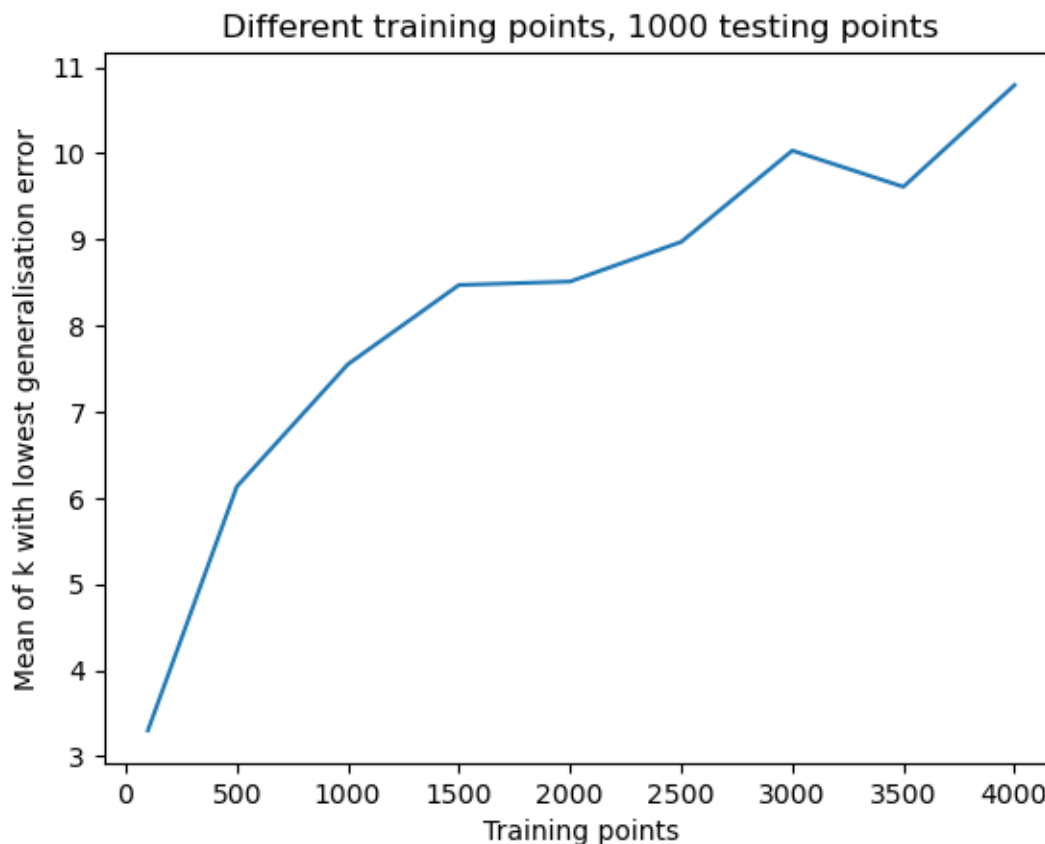


7 Q7



- The generalisation errors decrease first with k and then increase
- With increasing k , we have more neighbours for the majority votes and provide more accuracy
- But if k is too large, we potentially overfit the model, making generalisation error go up
- The generalisation errors go down with k quickly, but increase slowly
- For even k , it could have the undefined result with tie in majority vote

8 Q8



- With more training points, there are more dense points in the optimal circle, and hence larger k .
- The relationship of optimal k has roughly log relationship of training points.
- This corresponds to Bayesian information criterion: $BIC = -2\log(L) + p\log(N)$ where $\log(L)$ is the log likelihood of data, we have 1000 testing data for each k . We can assume this term is roughly same across different trainings. N is the number of training points.
- BIC is derived from Laplace approximation of posterior distribution with prior knowledge. Here we assume the training data are uniformly distributed a priori.

8.0.1 Corner case

- The corner case is when k is even and there is a tie in the majority vote
- In my function I use `Counter(lst).most_common(1)[0][0]` which returns the most common one randomly if there is a tie.

9 Q9

9.1 (a)

K_c is a kernel iff $c \geq 0$. * When $c \geq 0$, we can define a feature map $\phi(\mathbf{x}) := [\sqrt{c}, x_1, \dots, x_n]$ such that $K_c(\mathbf{x}, \mathbf{z}) = \langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle$. * If $c < 0$, we now need to show given any vector $\mathbf{a}$, $\mathbf{a}^T K_c \mathbf{a} \geq 0$:

$$\begin{aligned}
 \mathbf{a}^T K_c \mathbf{a} &= \sum_i^n \sum_j^n a_i a_j K_c(\mathbf{x}_i, \mathbf{x}_j) \\
 &= \sum_i \sum_j a_i a_j (c + \mathbf{x}_i^T \mathbf{x}_j) \\
 &= \sum_i \sum_j a_i a_j c + \sum_i \sum_j a_i a_j \mathbf{x}_i^T \mathbf{x}_j \\
 &= \|\mathbf{a}\|^2 c + \sum_i \sum_j a_i \mathbf{x}_i^T \mathbf{x}_j a_j \\
 &= \|\mathbf{a}\|^2 c + \left\| \sum_i a_i \mathbf{x}_i \right\|^2
 \end{aligned}$$

Here $\|\mathbf{a}\|^2$ and $\|\sum_i a_i \mathbf{x}_i\|^2$ are greater or equal to 0. It is guaranteed that $\mathbf{a}^T K_c \mathbf{a} \geq 0$ when $c \geq 0$. Notice when $c < 0$, there exists vector α s.t. $\mathbf{a}^T K_c \mathbf{a} < 0$, such α can be construct by a unit vector orthogonal to vector x .

9.2 (b)

Recall in kernel regression we have

$$K_c \alpha = \mathbf{y}$$

That is

$$c \mathbf{1}_n \mathbf{1}_n^T \alpha + K \alpha = \mathbf{y}$$

where K is the original state space of $\mathbf{x}$.

This is essentially acting as adding a bias to the regression, making the regression line have nonzero intercept and bring down the slope of the regression.

We can use a simple example to illustrate.

Suppose we only have 2 points: (x_1, y_1) and (x_2, y_2) . We have:

$$\begin{bmatrix} c + x_1^2 & c + x_1 x_2 \\ c + x_1 x_2 & c + x_2^2 \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \alpha_2 \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}$$

$$\begin{cases} c(\alpha_1 + \alpha_2) + x_1(\alpha_1 x_1 + \alpha_2 x_2) = y_1 \\ c(\alpha_1 + \alpha_2) + x_2(\alpha_1 x_1 + \alpha_2 x_2) = y_2 \end{cases}$$

$$f(x) = \alpha_1(c + x_1 x) + \alpha_2(c + x_2 x) = (\alpha_1 x_1 + \alpha_2 x_2)x + c(\alpha_1 + \alpha_2)$$

When c increases, α_1 and α_2 decrease. So the slope decreases. On the other hand, $c(\alpha_1 + \alpha_2)$ increases so the intercept increases.

10 Q10

In kernel regression, we have:

$$\alpha = K^{-1}\mathbf{y}$$

where K is the gram kernel matrix of all data points.

$$f(\mathbf{t}) = \alpha^\top K(\mathbf{x}, \mathbf{t}) = \mathbf{y}^\top K^{-1}K(\mathbf{x}, \mathbf{t})$$

Note $\mathbf{d} := K^{-1}K(\mathbf{x}, \mathbf{t})$ is a vector of size n with $\mathbf{d}_i = \sum_j^n K_{ij}^{-1} \exp(-\beta \|\mathbf{x}_j - \mathbf{t}\|^2)$

If we would like to simulate a 1-Nearest Neighbour Classifier and without loss of generality we let $\mathbf{y}_i$ is the nearest point to $\mathbf{t}$. In that case:

$$\text{sign}(f(\mathbf{t})) = \text{sign}(\mathbf{y}^\top \mathbf{d}) = \text{sign}(\mathbf{y}_i \mathbf{d}_i + \sum_{j \neq i}^n \mathbf{y}_j \mathbf{d}_j)$$

A sufficient condition would be $\mathbf{d}_i > 0$ and $\mathbf{d}_i > \sum_{j \neq i}^n |\mathbf{d}_j|$

11 Q11

11.1 (a)

$$\varepsilon_{\rho_i}(f_i) = \int_{\mathcal{C} \times \mathcal{Y}} \mathbb{I}\{f_i(x) \neq y\} d\rho_i(\{x, y\}) = 0$$

$$\varepsilon_{\rho_i}(f) = \int_{\mathcal{C} \times \mathcal{Y}} \mathbb{I}\{f(x) \neq y\} d\rho_i(\{x, y\}) \geq 0$$

where $f_i \in \mathcal{Y}^{\mathcal{C}}$

The first equality holds because we define the distribution of ρ_i in the way that $y = f_i(x)$

So we have

$$\varepsilon_{\rho_i}(f_i) = \inf_{f: \mathcal{X} \rightarrow \mathcal{Y}} \varepsilon_{\rho_i}(f) = 0$$

11.2 (b)

$$\mathbb{E}_{S \sim \rho_i^n} \varepsilon_{\rho_i}(A(S)) = \sum_{j=1}^k \varepsilon_{\rho_i}(A(S_j^i)) \times \frac{1}{k}$$

because every data set S_j is uniformly distributed in C^n .

Now we have:

$$\begin{aligned}
\max_i \mathbb{E}_{s \sim \rho_i^n} \varepsilon_{\rho_i}(A(S)) &= \max_i \frac{1}{k} \sum_{j=1}^k \varepsilon_{\rho_i}(A(S_j^i)) \\
(\text{max is larger than average}) &\geq \frac{1}{T} \sum_i \frac{1}{k} \sum_{j=1}^k \varepsilon_{\rho_i}(A(S_j^i)) \\
&= \frac{1}{k} \sum_{j=1}^k \frac{1}{T} \sum_i \varepsilon_{\rho_i}(A(S_j^i)) \\
&\geq \min_j \frac{1}{T} \sum_i \varepsilon_{\rho_i}(A(S_j^i))
\end{aligned}$$

11.3 (c)

$$\begin{aligned}
\frac{1}{T} \sum_{i=1}^T \varepsilon_p(A(S_j^i)) &= \frac{1}{T} \sum_{i=1}^T \int_{\mathcal{C} \times \mathcal{Y}} \mathbb{I}\{A(S_j^i) \neq y\} d\rho_i \\
&\geq \frac{1}{T} \sum_{i=1}^T \int_{R_j \times \mathcal{Y}} \mathbb{I}\{A(S_j^i)(v) \neq f_i(v)\} d\rho_i(\{(x, y)\}) \\
&= \frac{1}{T} \sum_{i=1}^T \sum_{m=1}^p \mathbb{I}\{A(S_j^i)(v_m) \neq f_i(v_m)\} \times \frac{1}{2n} \\
&= \frac{1}{2} \frac{1}{n} \sum_{m=1}^p \frac{1}{T} \sum_{i=1}^T \mathbb{I}\{A(S_j^i)(v_m) \neq f_i(v_m)\} \\
&\geq \frac{1}{2} \frac{1}{p} \sum_{m=1}^p \frac{1}{T} \sum_{i=1}^T \mathbb{I}\{A(S_j^i)(v_m) \neq f_i(v_m)\} \\
&\geq \frac{1}{2} \min_{v \in R_j} \frac{1}{T} \sum_{i=1}^T \mathbb{I}\{A(S_j^i)(v_m) \neq f_i(v_m)\}
\end{aligned}$$

The second last inequality holds as we take R_j as the subset of points of $\mathcal{C}$ that do not belong to S_j . Since $|S_j| \leq n$ and $|\mathcal{C}| = 2n$, we have $p \geq n$.

11.4 (d)

If we partition $\mathcal{Y}^C$ into $\frac{T}{2}$ pairs $(f_i, f_{i'})$ such that $f_i(x) \neq f_{i'}(x)$ and we know $f(x)$ can only take binary values (either 1 or -1), we have the following:

$$\mathbb{I}\{A(S_j^i)(v) \neq f_i(v)\} + \mathbb{I}\{A(S_j^{i'})(v) \neq f_{i'}(v)\} = 1$$

as exclusively only one can be 1.

And we have $\frac{T}{2}$ pairs, we sum $\frac{T}{2}$ twice to get.

$$\sum_{i=1}^T \mathbb{I} \{A(S_j^i)(v) \neq f_i(v)\} + \sum_{i=1}^T \mathbb{I} \{A(S_j^{i'})(v) \neq f_{i'}(v)\} = T$$

So

$$\frac{1}{T} \sum_{i=1}^T \mathbb{I} \{A(S_j^i)(v) \neq f_i(v)\} = \frac{1}{2}$$

11.5 (e)

Let $x = 1 - z \in [0, 1]$ $\mathbb{E}[x] = 1 - \mu$.

$$\begin{aligned} P(x > a) &= P(1 - z > a) = P(z < 1 - a) < \frac{1 - \mu}{a} \\ P(z > 1 - a) &= 1 - P(z < 1 - a) > \frac{a - 1 + \mu}{a} = \frac{\mu - (1 - a)}{a} \end{aligned}$$

11.6 (f)

$$\varepsilon_\rho(A(s)) \leq 1 \int_{\mathcal{X} \times \mathcal{Y}} d\rho(x, y) = 1$$

Hence $\varepsilon_p(A(S)) \in [0, 1]$

$$P_{s \sim \rho^n} \left(\varepsilon_\rho(A(S)) > \frac{1}{8} \right) \geq \frac{\mathbb{E}[\varepsilon_\rho(A(S))] - \frac{1}{8}}{\frac{7}{8}} = \frac{8\mathbb{E}[\varepsilon_\rho(A(S))] - 1}{7}$$

$$\begin{aligned} \mathbb{E}[\varepsilon_\rho(A(S))] &= \frac{1}{T} \sum_{i=1}^T \mathbb{E}_{S \sim \rho_i^n} \varepsilon_{\rho_i}(A(S)) \geq \min_j \frac{1}{T} \sum_{i=1}^T \varepsilon_{\rho_i}(A(S_j^i)) \\ &\geq \frac{1}{2} \min_v \frac{1}{T} \sum_{i=1}^T \mathbb{I} \{A(S_j^i)(v) \neq f_i(v)\} = \frac{1}{4} \end{aligned}$$

Hence

$$P_{S \sim \rho^n} \left(\varepsilon_\rho(A(S)) > \frac{1}{8} \right) \geq \frac{1}{7}$$

11.7 (g)

We consider all possible functions in the space without any bias or assumptions.

1. For a function f_i s.t. $\varepsilon_{\rho_i}(f_i) = 0$ on a subset, the probability of that function mis-classification approximation error on the universe larger than $\frac{1}{8}$ is more than $\frac{1}{7}$.

2. With the previous setting, we have

$$P_{S \sim \rho^n} (\varepsilon_\rho(A(S)) \leq \epsilon) \geq 1 - \delta$$

This implies

$$P_{S \sim \rho^n} (\varepsilon_\rho(A(S)) > \epsilon) \leq \delta \quad \forall \epsilon, \delta \in (0, 1)$$

This contradicts to No-Free-Lunch theorem.

If we do not make any assumption of the function space, No-Free-Lunch theorem implies the space $\mathcal{Y}^{\mathcal{X}}$ of all functions $f : \mathcal{X} \rightarrow \mathcal{Y}$ is NOT learnable. Because of $P_{S \sim \rho^n} (\varepsilon_\rho(A(S)) > \frac{1}{8} - 0) \geq \frac{1}{7}$, it states no matter how many countable n we have, there is always non zero probability that approximation error is larger than $1/8$.

3.

- A certain algorithm can perform well on specific types of problems but may perform poorly on the others. We need to tailor the algorithms to the specific structure and characteristics of a given problem domain. We should try a variety of algorithms when approaching a new problem.
- There is no universal solver for all machine learning tasks. The model we train on the specific datasets could struggle to generalise well to out-of-sample datasets unless they share very similar characteristics. We should make sure training data and testing data have same characteristics.
- We need to make assumptions and narrow down the function space we would like to learn